

5-ma'ruza. Abstrakt sinflar

Reja:

1. Abstrakt sinflar
2. sealed kalit so'zi bilan vorislikni oldini olish
3. object sinfi umumiy ma'lumotlar turi sifatida

Ba'zida barcha voris olingan sinflar uchun umumiy shaklni belgilaydigan va uning a'zolarini har bir sinfga voris qilib beradigan tayanch sinf yaratishga to'g'ri keladi. Bunday sinfda faqat metodlarning vazifasi aniqlanadi, ular tayanch sinfning o'zida emas, balki voris sinflarda bajarilishi kerak. Vorislik mavzusida keltirilgan misolda Tekislik sinfida ko'rsatilgan Yuza() metodi oddiygina to'ldiruvchi sifatida belgilangan edi. Ushbu metod har qanday turdagi ikki o'lchovli obyektning yuzasini hisoblamaydi yoki ko'rsatmaydi.

Bu yerda, metod ba'zi hollarda uning tayanch sinfda barcha hollar uchun o'rinli emas. Bu ikkita yo'l bilan yechiladi. Birinchi yo'l, oldingi misoldagidek, oddiy xabar chiqarish orqali. Metodlar tayanch sinfda e'lon qilinishi mumkin, bu sinf ma'noga ega bo'lishi uchun voris sinfda bekor qilinishi kerak. Masalan, Uchburchak sinfini olaylik. Agar Yuza() metodi bekor qilinmasa, u to'liq bo'lmaydi. Bunday hollarda voris sinf barcha kerakli metodlarni bekor qilishi uchun yo'l boshqacha yo'ldan foydalaniladi. Bu ikkinchi yo'l abstrakt metoddan foydalanish hisoblanadi.

abstract turidagi o'zgaruvchi bilan abstrakt metod yaratiladi. Abstrakt metod o'z tanasiga ega emas va shuning uchun tayanch sinfda amalga oshirilmaydi. Uni voris sinfda bekor qilish kerak, chunki uning tayanch sinfi foydalanishga yaroqsiz. Abstrakt metod avtomatik ravishda virtual metodga aylanadi. Aslida, *virtual* va *abstract* modifikatorlardan birgalikda foydalanilmaydi.

Abstrakt metodni aniqlash uchun quyidagi umumiy shakldan foydalaniladi.

abstract tur Nom(Parametrlar);

Ko'rib turganingizdek, abstrakt metodning tanasi yo'q. *abstract* modifikator statik metodlarda emas, faqat metod nusxalarida ishlatilishi mumkin. Indeksatorlar va xossalar ham abstrakt bo'lishi mumkin.

Bir yoki bir nechta abstrakt metodlarni o'z ichiga olgan sinf ham *abstract* deb e'lon qilinishi kerak va buning uchun *abstract* modifikator sinf e'lonidan oldin ko'rsatiladi hamda abstrakt sinf to'liq aniqlanmaganligi sababli u obyektlarga ega bo'lishi mumkin emas. Shuning uchun *new* operatori yordamida abstrakt sinf obyektini yaratishga urinish kompilyatsiya vaqtida xatolikka olib keladi.

Biror bir sinf abstrakt sinfni voris qilib olsa, u tayanch sinfning barcha abstrakt metodlarini amalga oshirishi kerak. Aks holda, voris sinf ham abstrakt sifatida belgilanishi kerak. Abstrakt atribut sinfning to'liq amalga oshirilishiga erishilgunga qadar voris qilib olinadi.

Abstrakt sinfdan foydalanib, oldingi mavzuda berilgan masaladagi Tekislik sinfini kengaytirish mumkin. Quyida joylashgan Tekislik sinfida Yuza() metodi va Tekislik sinfning o'zi *abstract* deb e'lon qilinadi. Bu Tekislik sinfdan olingan barcha voris sinflar Yuza() metodini bekor qilishi kerakligini anglatadi.

```
// Abstrakt sinf yaratish
using System;
abstract class Tekislik
{
    double a;
    double h;
    // Standart konstruktor
    public Tekislik()
    {
        Asos = Balandlik = 0.0;
        nom = "null";
    }
}
```

```

// Parametrli konstruktor
public Tekislik(double w, double h, string n)
{
    Asos = w;
    Balandlik = h;
    nom = n;
}
// Bir xil asos va balandlikdagi konstruktor
public Tekislik(double x, string n)
{
    Asos = Balandlik = x;
    nom = n;
}
// Tekislik obyektining nusxasi
public Tekislik(Tekislik obyekt)
{
    Asos = obyekt.Asos;
    Balandlik = obyekt.Balandlik;
    nom = obyekt.nom;
}
// Obyektning asosi va balandligi xossalari
public double Asos
{
    get { return a; }
    set { a = value < 0 ? -value : value; }
}
public double Balandlik
{
    get { return h; }
    set { h = value < 0 ? -value : value; }
}
public string nom { get; set; }
public void Show()
{
    Console.WriteLine("Asos va Balandlik: " +
        Asos + " va " + Balandlik);
}
// Endi Yuza() metodi abstrakt
public abstract double Yuza();
}
// Tekislik sinfidan voris olingan Uchburchak sinfi
class Uchburchak : Tekislik
{
    string UchburchakTuri;
    // Standart konstruktor
    public Uchburchak()
    {
        UchburchakTuri = "null";
    }
    //Uchburchak sinfi uchun konstruktor
    public Uchburchak(string s, double w, double h) : base(w, h, "uchburchak")
    {

```

```

        UchburchakTuri = s;
    }
    // Teng yonli uchburchak
    public Uchburchak(double x) : base(x, "uchburchak")
    {
        UchburchakTuri = "teng yonli";
    }
    // Uchburchak turidagi obyektning nusxasi
    public Uchburchak(Uchburchak obyekt) : base(obyekt)
    {
        UchburchakTuri = obyekt.UchburchakTuri;
    }
    // Uchburchak sinfi uchun Yuza() metodini bekor qilish
    public override double Yuza()
    {
        return Asos * Balandlik / 2;
    }
    // Uchburchak turini ko'rsatish
    public void ShowUchburchakTuri()
    {
        Console.WriteLine("Uchburchak: " + UchburchakTuri);
    }
}
// Tekislik sinfidan voris olingan Turtburchak sinfi
class Turtburchak : Tekislik
{
    // Turtburchak sinfi uchun konstruktor
    public Turtburchak(double w, double h) :
        base(w, h, "to'g'ri to'rtburchak")
    { }
    // Kvadrat konstruktori
    public Turtburchak(double x) :
        base(x, "to'g'ri to'rtburchak")
    { }
    // Turtburchak turidagi obyektning nusxasi
    public Turtburchak(Turtburchak obyekt) : base(obyekt) { }

    // Agar to'rtburchak kvadrat bo'lsa, true qiymatini qaytaradi
    public bool Kvadrat()
    {
        if (Asos == Balandlik) return true;
        return false;
    }
    // Turtburchak sinfi uchun Yuza() metodini bekor qilish
    public override double Yuza()
    {
        return Asos * Balandlik;
    }
}
class AbsShakl
{
    static void Main()

```

```

{
    Tekislik[] shakllar = new Tekislik[4];
    shakllar[0] = new Uchburchak("to'g'ri burchakli", 8.0, 12.0);
    shakllar[1] = new Turtburchak(10);
    shakllar[2] = new Turtburchak(10, 4);
    shakllar[3] = new Uchburchak(7.0);
    for (int i = 0; i < shakllar.Length; i++)
    {
        Console.WriteLine("Obyekt - " + shakllar[i].nom);
        Console.WriteLine("Yuza: " + shakllar[i].Yuza());
        Console.WriteLine();
    }
    Console.ReadKey();
}
}

```

Dastur natijasi quyidagicha:

Obyekt - uchburchak

Yuza: 48

Obyekt - to'g'ri to'rtburchak

Yuza: 100

Obyekt - to'g'ri to'rtburchak

Yuza: 40

Obyekt - uchburchak

Yuza: 24,5

Yuqoridagi dastur kodidan ko'rinib turibdiki, barcha voris sinflarda Yuza() metodi bekor qilingan va *abstract* deb e'lon qilingan. Yuza() metodini bekor qilmaydigan voris sinf yaratilgan. Bu yerda, kompilyatsiya paytida xatolik kelib chiqadi. Tekislik turidagi obyektlarga havolalar yaratish imkoniyati ham mavjud va bu yuqoridagi dasturda amalga oshirildi, lekin Tekislik turidagi obyektlarni e'lon qilib bo'lmaydi. Shuning uchun shakllar massivi Main() metodida 4 ta elementga qisqartiriladi va umumiy ikki o'lchovli shakl uchun Tekislik turidagi obyekt yaratilmaydi.

Tekislik sinfi ham Show() metodini o'z ichiga oladi va u *abstract* modifikator bilan e'lon qilinmaydi. Abstrakt sinflar uchun asl shaklda qo'llanilishi mumkin bo'lgan metodlarni voris sinfga kiritish ma'qul va faqat *abstract* deb e'lon qilingan metodlar voris sinflarda e'lon qilinishi kerak.

Quyidagi dastur kodida abstrakt sinfni namoyish etishga oid yana bitta misol keltirilgan:

```

using System;
namespace AbstraktSinf {
    abstract class Shakl {
        public abstract int area();
    }
    class Turtburchak: Shakl {
        private int length;
        private int width;
        public Turtburchak( int a = 0, int b = 0) {
            length = a;
            width = b;
        }
        public override int area () {

```

```

        Console.WriteLine("To'rtburchak sinf maydoni :");
        return (width * length);
    }
}
class TurtburchakTester {
    static void Main(string[] args) {
        Turtburchak r = new Turtburchak(10, 7);
        double a = r.area();
        Console.WriteLine("Maydon: {0}",a);
        Console.ReadKey();
    }
}
}

```

Yuqoridagi kod kompilyatsiya qilinganda quyidagi natijani beradi:

To'rtburchak sinf maydoni:

Maydon: 70

sealed kalit so'zi bilan vorislikni oldini olish

Vorislikning samaradorligiga qaramasdan, ba'zida uning oldini olish zarurati tug'iladi. C# da *sealed* kalit so'zidan foydalanib, sinfni voris qilib olishni cheklash mumkin.

Sinfni voris qilib olishning oldini olish uchun sinfdan oldin *sealed* kalit so'zni ko'rsatish kifoya. Sinfni ham *abstract*, ham *sealed* deb e'lon qilishga ruxsat berilmaydi, chunki abstrakt sinfnining o'zi to'liq amalga oshirilmagan va to'liq amalga oshirishni ta'minlash uchun bu borada voris sinflarga tayanadi.

Quyida *sealed* turidagi sinfni e'lon qilish misoli keltirilgan.

```

sealed class A
{
    // ...
}
// Quyidagi sinf yaroqsiz hisoblanadi
class B : A
{
    // XATO! A sinfdan voris olib bo'lmaydi
    // ...
}

```

Yuqoridagi kodda izohlardan ko'rinib turibdiki, B sinf A sinfdan voris ololmaydi, chunki sinf *sealed* deb e'lon qilingan.

Eslatma: *sealed* kalit so'zi virtual metodlarda ham qo'llanilishi mumkin, bu ularning keyingi aniqlanishiga yo'l qo'ymaydi. Faraz qilaylik, asosiy B sinf va voris D sinf mavjud. B sinfda *virtual* deb e'lon qilingan metod D sinfda *sealed* deb e'lon qilinishi mumkin. Shu sababli, sinfdan voris bo'lgan har qanday sinfda bu metodni bekor qilishning oldi olinadi. Xuddi shunday holat quyidagicha bo'ladi.

```

class B
{
    public virtual void MyMethod() { /* ... */ }
}
class D : B
{
    // Bu MyMethod() metodini yopadi va uni keyingi bekor qilinishini oldini oladi.
    sealed public override void MyMethod() { /* ... */ }
}
class X : D
{

```

```
// Xato! MyMethod() metodi bekor qilingan!
public override void MyMethod() { /* ... */ }
}
```

MyMethod() metodi D sinfida yopilgan, shuning uchun X sinfida bekor qilib bo'lmaydi.

object sinfi umumiy ma'lumotlar turi sifatida

Agar *object* barcha boshqa turlar uchun tayanch sinf bo'lsa va avtomatik ravishda turli xil qiymatlar jamlangan bo'lsa, *object* sinfidan "umumiy" berilganlar turi sifatida foydalanish mumkin. Misol uchun avval *object* turidagi massivni yaratadigan va keyin uning elementlariga har xil berilganlar turlarining qiymatlarini aniqlaydigan quyidagi dastur kodini ko'rib chiqamiz.

```
// object turidagi massiv yaratish
using System;
class GenericDemo
{
    static void Main()
    {
        object[] a = new object[10];
        //Butun qiymat berish
        for (int i = 0; i < 3; i++)
            a[i] = i;
        // Haqiqiy qiymat berish.
        for (int i = 3; i < 6; i++)
            a[i] = (double)i / 2;
        // satr, belgi va mantiqiy qiymat berish
        a[6] = "Satr1";
        a[7] = true;
        a[8] = 'A';
        a[9] = "Satr2";
        for (int i = 0; i < a.Length; i++)
            Console.WriteLine("a[" + i + "]: " + a[i] + " ");
    }
}
```

Ushbu misoldan ko'rinib turibdiki, *object* sinfining obyektiga havola orqali istalgan turdagi ma'lumotlarga kirish mumkin, chunki *object* mos yozuvlar turi o'zgaruvchisi boshqa barcha turdagi ma'lumotlarga havolani saqlashi mumkin. Shu sababli ushbu misoldagi obyekt turidagi massivda deyarli har qanday turdagi ma'lumotlar saqlanadi. *object* sinfining obyektlariga havolalar bilan stek sinfini osongina yaratish mumkin. Bu har qanday turdagi ma'lumotlarni stekda saqlashga imkon beradi.

Eng muhimi, C# ning 2.0 versiyasidan boshlab, dasturlash uchun umumiy berilganlar turlari, *generics* turi yaratildi. *generics* ning joriy etilishi har xil turdagi berilganlarni avtomatik ravishda qayta ishlaydigan sinflar va algoritmlarni aniqlashni osonlashtirdi. *generics* yangi kod yaratishda *object* sinfidan umumiy havola turi sifatida foydalanish zaruratini yo'q qildi.